11249A250.exp_1.py

```python
import numpy as np
a = np.array([1, 2, 3, 4, 5])
b = np.arange(1, 10, 2)
c = np.linspace( start: 0,  stop: 1,  num: 5)
d = np.zeros((2, 3))
e = np.ones((2, 2))
f = np.eye(3)
g = np.random.randint( low: 1,  high: 100, size=(3, 3))
print("Array a:", a)
print("Arange b:", b)
print("Linspace c:", c)
print("Zeros d:\n", d)
print("Ones e:\n", e)
print("Identity f:\n", f)
print("Random g:\n", g)
print("\nShape:", g.shape)
print("Dimensions:", g.ndim)
print("Size:", g.size)
print("Data type:", g.dtype)
reshaped = g.reshape(1, 9)
print("\nReshaped array:", reshaped)
print("Transpose:\n", g.T)
print("Flattened:", g.flatten())
print("\nAdd:", np.add(a, 2))
print("Multiply:", np.multiply(a, 3))
print("Square Root:", np.sqrt(a))
print("Power:", np.power(a, 2))
print("Absolute:", np.abs([-1, -2, 3]))
print("\nMean:", np.mean(a))
print("Median:", np.median(a))
print("Standard Deviation:", np.std(a))
print("Variance:", np.var(a))
print("Minimum:", np.min(a))
print("Maximum:", np.max(a))
print("Sum:", np.sum(a))
print("Percentile (50):", np.percentile(a,  q: 50))
```

11249A250.exp_1.py

```python
print("Median:", np.median(a))
print("Standard Deviation:", np.std(a))
print("Variance:", np.var(a))
print("Minimum:", np.min(a))
print("Maximum:", np.max(a))
print("Sum:", np.sum(a))
print("Percentile (50):", np.percentile(a, q: 50))
print("\nColumn-wise Sum:\n", np.sum(g, axis=0))
print("Row-wise Mean:\n", np.mean(g, axis=1))
print("\nFirst element:", a[0])
print("Slice:", a[1:4])
print("2D Slice:\n", g[0:2, 1:3])
x = np.array([[1, 2], [1, 4]])
y = np.array([[5, 6], [7, 8]])
print("\nMatrix x:\n", x)
print("Matrix y:\n", y)
print("\nDot Product:\n", np.dot(x, y))
print("Matrix Multiplication:\n", np.matmul(x, y))
print("Determinant:", np.linalg.det(x))
print("Inverse:\n", np.linalg.inv(x))
np.random.seed(1)
rand_sample = np.random.choice(a, size=3)
print("\nRandom Choice:", rand_sample)
h1 = np.array([1, 2, 3])
h2 = np.array([4, 5, 6])
print("\nConcatenate:", np.concatenate((h1, h2)))
print("Vertical Stack:\n", np.vstack((h1, h2)))
print("Horizontal Stack:", np.hstack((h1, h2)))
np.save( file: "sample_array.npy", a)
loaded = np.load("sample_array.npy")
print("\nLoaded Array:", loaded)
```

11249A250.exp_1.py

Run scratch_3

```
"C:\Users\scsvm\PycharmProjects 11249A279\pythonProject\.venv\Scripts\python.exe" C:\Users\scsvm\AppDa
Array a: [1 2 3 4 5]
Arange b: [1 3 5 7 9]
Linspace c: [0.   0.25 0.5  0.75 1.  ]
Zeros d:
 [[0. 0. 0.]
 [0. 0. 0.]]
Ones e:
 [[1. 1.]
 [1. 1.]]
Identity f:
 [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
Random g:
 [[28 54 76]
 [29 58 11]
 [59 79 65]]

Shape: (3, 3)
Dimensions: 2
Size: 9
Data type: int32

Reshaped array: [[28 54 76 29 58 11 59 79 65]]
Transpose:
 [[28 29 59]
 [54 58 79]
 [76 11 65]]
Flattened: [28 54 76 29 58 11 59 79 65]

Add: [3 4 5 6 7]
Multiply: [ 3  6  9 12 15]
Square Root: [1.         1.41421356 1.73205081 2.         2.23606798]
Power: [ 1  4  9 16 25]
Absolute: [1 2 3]
```

11249A250.exp_1.py

Run scratch_3

```
Square Root: [1.         1.41421356 1.73205081 2.         2.23606798]
Power: [ 1  4  9 16 25]
Absolute: [1 2 3]

Mean: 3.0
Median: 3.0
Standard Deviation: 1.4142135623730951
Variance: 2.0
Minimum: 1
Maximum: 5
Sum: 15
Percentile (50): 3.0

Column-wise Sum:
 [116 191 152]
Row-wise Mean:
 [52.66666667 32.66666667 67.66666667]

First element: 1
Slice: [2 3 4]
2D Slice:
 [[54 76]
 [58 11]]

Matrix x:
 [[1 2]
 [1 4]]
Matrix y:
 [[5 6]
 [7 8]]

Dot Product:
 [[19 22]
 [33 38]]
Matrix Multiplication:
 [[19 22]
```

11249A250.exp_1.py

Run scratch_3

```
First element: 1
Slice: [2 3 4]
2D Slice:
 [[54 76]
 [58 11]]

Matrix x:
 [[1 2]
 [1 4]]
Matrix y:
 [[5 6]
 [7 8]]

Dot Product:
 [[19 22]
 [33 38]]
Matrix Multiplication:
 [[19 22]
 [33 38]]
Determinant: 2.0
Inverse:
 [[ 2.  -1. ]
 [-0.5  0.5]]

Random Choice: [4 5 1]

Concatenate: [1 2 3 4 5 6]
Vertical Stack:
 [[1 2 3]
 [4 5 6]]
Horizontal Stack: [1 2 3 4 5 6]

Loaded Array: [1 2 3 4 5]

Process finished with exit code 0
```

ches > 11249A250.exp_1.py

EXP NO :1.2

11249A250.exp_1.py 11249a250.exp_2.py

```python
import pandas as pd
data = {

    'Name': ['Alice', 'Bob', 'Charlie'],

    'Age': [25, 30, 35],

    'City': ['New York', 'Paris', 'London']
}
df = pd.DataFrame(data)

print(df.head())

print(df.info())

print(df.describe())

print('B.Selection and Filtering')

ages = df['Age']

above_25 = df[df['Age'] > 25]

row_0 = df.iloc[0]

specific_val = df.loc[0, 'City']

print("C.DataCleaning(HandlingMissingValues)")

print(df.isnull().sum())

df_clean = df.dropna()

df_filled = df.fillna(0)  # Fill NaNs with 0
```

OUTPUT:

```
11249A250.exp_1.py        11249a250.exp_2.py
Run    11249a250.exp_2

"C:\Users\scsvm\PycharmProjects 11249A279\pythonProject\.venv\Scripts\python.exe" C:\Users\scsvm\AppData\Ro
      Name  Age      City
0    Alice   25  New York
1      Bob   30     Paris
2  Charlie   35    London
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    3 non-null      object
 1   Age     3 non-null      int64
 2   City    3 non-null      object
dtypes: int64(1), object(2)
memory usage: 200.0+ bytes
None
        Age
count   3.0
mean   30.0
std     5.0
min    25.0
25%    27.5
50%    30.0
75%    32.5
max    35.0
B.Selection and Filtering
C.DataCleaning(HandlingMissingValues)
Name    0
Age     0
City    0
dtype: int64

Process finished with exit code 0
```

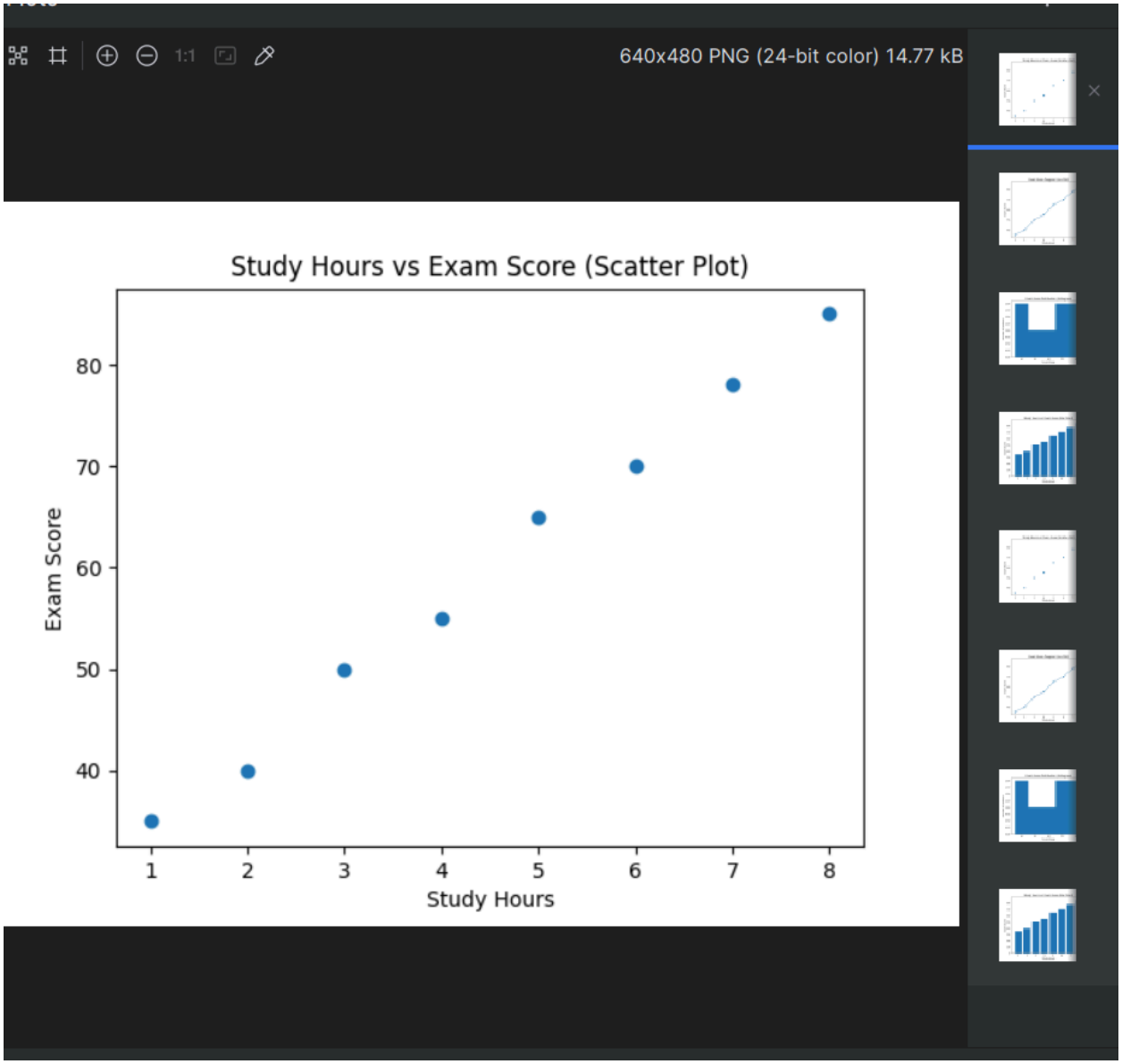
EXP NO:1.3

11249A250_exp1.3.py

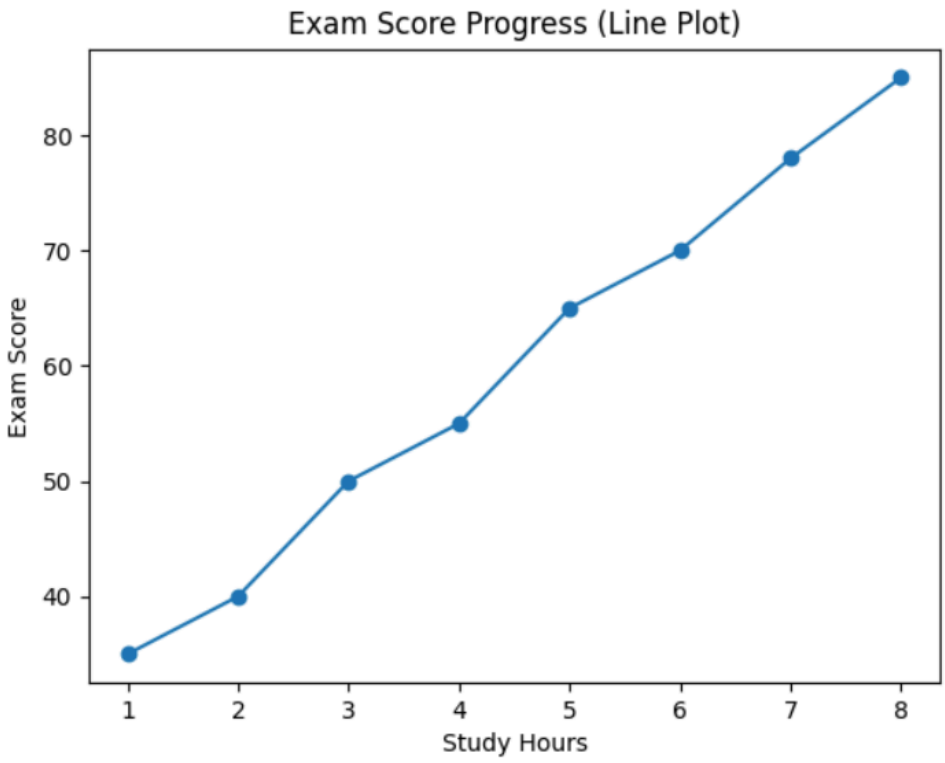
```python
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
data = {'StudyHours': [1, 2, 3, 4, 5, 6, 7, 8],
'ExamScore': [35, 40, 50, 55, 65, 70, 78, 85]}
df = pd.DataFrame(data)
# 1 scatter Plot
plt.scatter(df['StudyHours'], df['ExamScore'])
plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Study Hours vs Exam Score (Scatter Plot)")
plt.show()
# 2 Line Plot
plt.plot( *args: df['StudyHours'], df['ExamScore'], marker='o')
plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Exam Score Progress (Line Plot)")
plt.show()
# 3 Histogram
plt.hist(df['ExamScore'], bins=5)
plt.xlabel("Score Range")
plt.ylabel("Number of Students")
plt.title("Exam Score Distribution (Histogram)")
plt.show()
# 4 Bar Chart
plt.bar(df['StudyHours'], df['ExamScore'])
plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Study Hours vs Exam Score (Bar Chart)")
plt.show()
```

11249A250_exp1.3

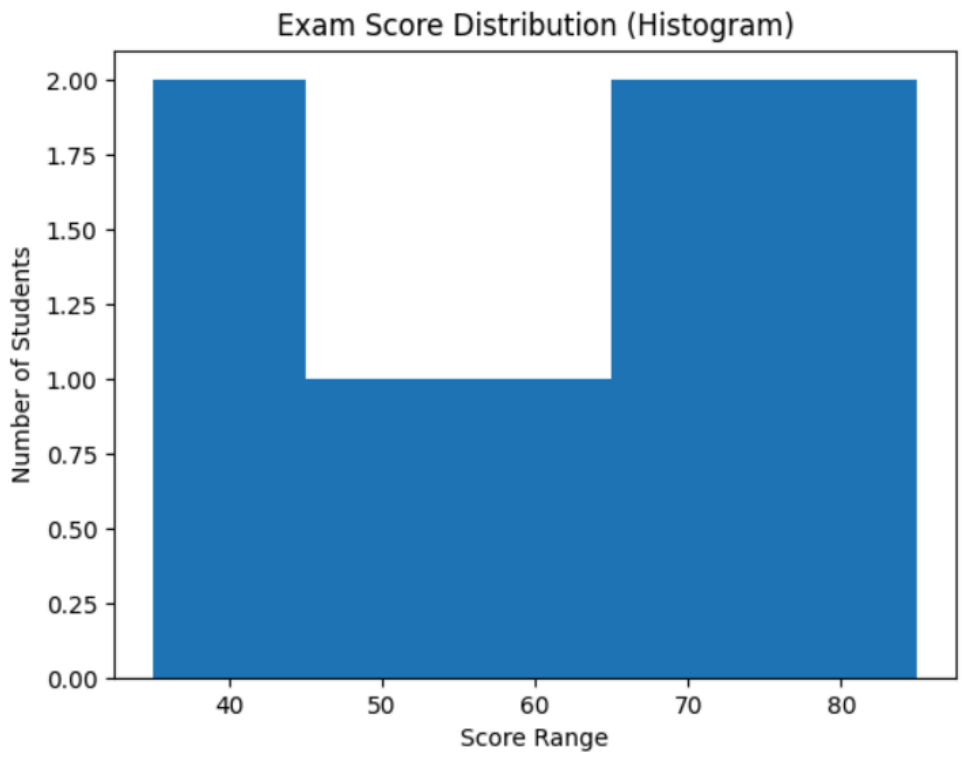
OUTPUT:


640x480 PNG (24-bit color) 14.77 kB
Study Hours vs Exam Score (Scatter Plot)
Exam Score
80
70
60
50
40
1
2
3
4
5
6
7
8
Study Hours

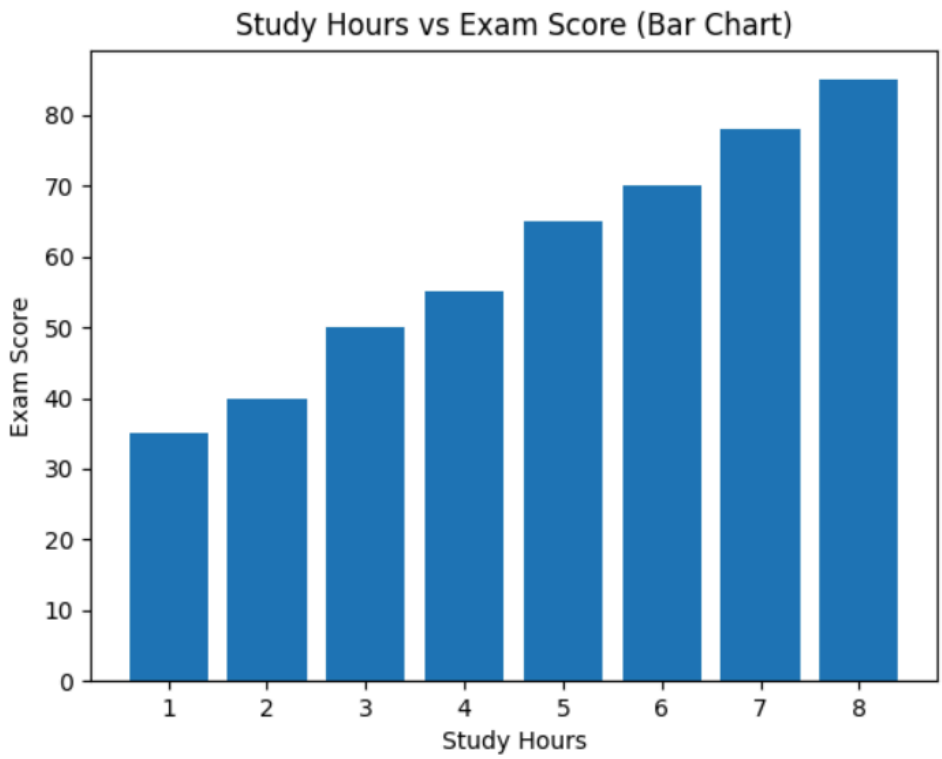
640x480 PNG (24-bit color) 19.17 kB



640x480 PNG (24-bit color) 21.16 kB



640x480 PNG (24-bit color) 25.97 kB



EXP NO:2

11249A250_exp1.3.py 11249A250_expno2.py

```python
import pandas as pd

import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
df = pd.read_csv('/content/data_output.csv')
print("--- Original raw data ---")
print(df)
print("\n--- Missing values count ---")
print(df.isnull().sum())
df['Age'] = df['Age'].fillna(df['Age'].mode()[0])
df['Salary'] = df['Salary'].fillna(df['Salary'].mode()[0])
print("\n--- Data after cleaning ---")
print(df)
df_encoded = pd.get_dummies(df, columns=['Country'])
df_encoded['Purchased'] = df_encoded['Purchased'].map({'Yes': 1, 'No': 0})
print("\n--- Data after encoding ---")
print(df_encoded)
X = df_encoded.drop( labels: 'Purchased', axis=1)
y = df_encoded['Purchased']
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print("\n--- Final processed data (ready for AI) ---")
print(pd.DataFrame(X_scaled, columns=X.columns).head())
```

OUTPUT:

```
    0   India  22.0  40000.0        No
    1     USA  25.0  60000.0       Yes
... 2   India   NaN  50000.0       Yes
    3     USA  30.0      NaN        No
    4      UK  28.0  72000.0       Yes
    5   India  35.0  58000.0       Yes


    ---missing values count---
    Country      0
    Age          1
    Salary       1
    Purchased    0
    dtype: int64


    ---DATA AFTER CLEANING---
      Country   Age   Salary Purchased
    0   India  22.0  40000.0        No
    1     USA  25.0  60000.0       Yes
    2   India  28.0  50000.0       Yes
    3     USA  30.0  60000.0        No
    4      UK  28.0  72000.0       Yes
    5   India  35.0  58000.0       Yes


    ---DATA AFTER ENCODING---
        Age   Salary  Purchased  Country_India  Country_UK  Country_USA
    0  22.0  40000.0          0           True       False        False
    1  25.0  60000.0          1          False       False         True
    2  28.0  50000.0          1           True       False        False
    3  30.0  60000.0          0          False       False         True
    4  28.0  72000.0          1          False        True        False
    5  35.0  58000.0          1           True       False        False


    ---FINAL PROCESSED DATA(Ready for ai)---
            Age    Salary  Country_India  Country_UK  Country_USA
    0 -1.484615 -1.693214            1.0   -0.447214    -0.707107
    1 -0.742307  0.338643           -1.0   -0.447214     1.414214
    2  0.000000 -0.677285            1.0   -0.447214    -0.707107
    3  0.494872  0.338643           -1.0   -0.447214     1.414214
    4  0.000000  1.557757           -1.0    2.236068    -0.707107
```

EXP NO:3

11249A250_exp1.3.py 11249A250_expno2.py 11249A250.expno-3.py

```python
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error
diabetes = datasets.load_diabetes()
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: diabetes.data, diabetes.target, test_size=0.2, random_state=42
)
lr = LinearRegression()
lr.fit(X_train, y_train)
ridge = Ridge(alpha=1.0)
ridge.fit(X_train, y_train)
lasso = Lasso(alpha=0.1)
lasso.fit(X_train, y_train)
lr_pred = lr.predict(X_test)
ridge_pred = ridge.predict(X_test)
lasso_pred = lasso.predict(X_test)
print("Linear Regression MSE:", mean_squared_error(y_test, lr_pred))
print("Ridge Regression MSE:", mean_squared_error(y_test, ridge_pred))
print("Lasso Regression MSE:", mean_squared_error(y_test, lasso_pred))
```

11249A250.expno-3

```
C:\Users\jyoth\PycharmProjects\pythonProject\.venv\Scripts\python.exe C:\Users\jyoth\App
Linear Regression MSE: 2900.1936284934814
Ridge Regression MSE: 3077.41593882723
Lasso Regression MSE: 2798.1934851697188

Process finished with exit code 0
```

EXP NO:4

11249A250_exp1.3.py | 11249A250_expno2.py | 11249A250.expno-3.py | 11249A250.expno-4.py

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
data = pd.read_csv("content/temp_hum_play_data.csv")
le = LabelEncoder()
data['Play'] = le.fit_transform(data['Play'])
X = data[['Temperature', 'Humidity']]
y = data['Play']
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: X, y, test_size=0.2, random_state=42
)
model = GaussianNB()
model.fit(X_train, y_train)
prediction = model.predict(X_test)
all_prediction = model.predict(X)
print("Accuracy:", accuracy_score(y_test, prediction))
print("\nClass Priors:\n", model.class_prior_)
print("\nMean (theta):\n", model.theta_)
print("\nVariance:\n", model.var_)
plt.figure(figsize=(8, 6))
plt.scatter(
    X['Temperature'], X['Humidity'],
    c=y, cmap='coolwarm', marker='o', label='Actual'
)
plt.scatter(
    X['Temperature'], X['Humidity'],
    c=all_prediction, cmap='coolwarm',
    marker='x', s=100, label='Predicted'
```

11249A250_exp1.3.py 11249A250_expno2.py 11249A250.expno-3.py 11249A250.expno-4.py

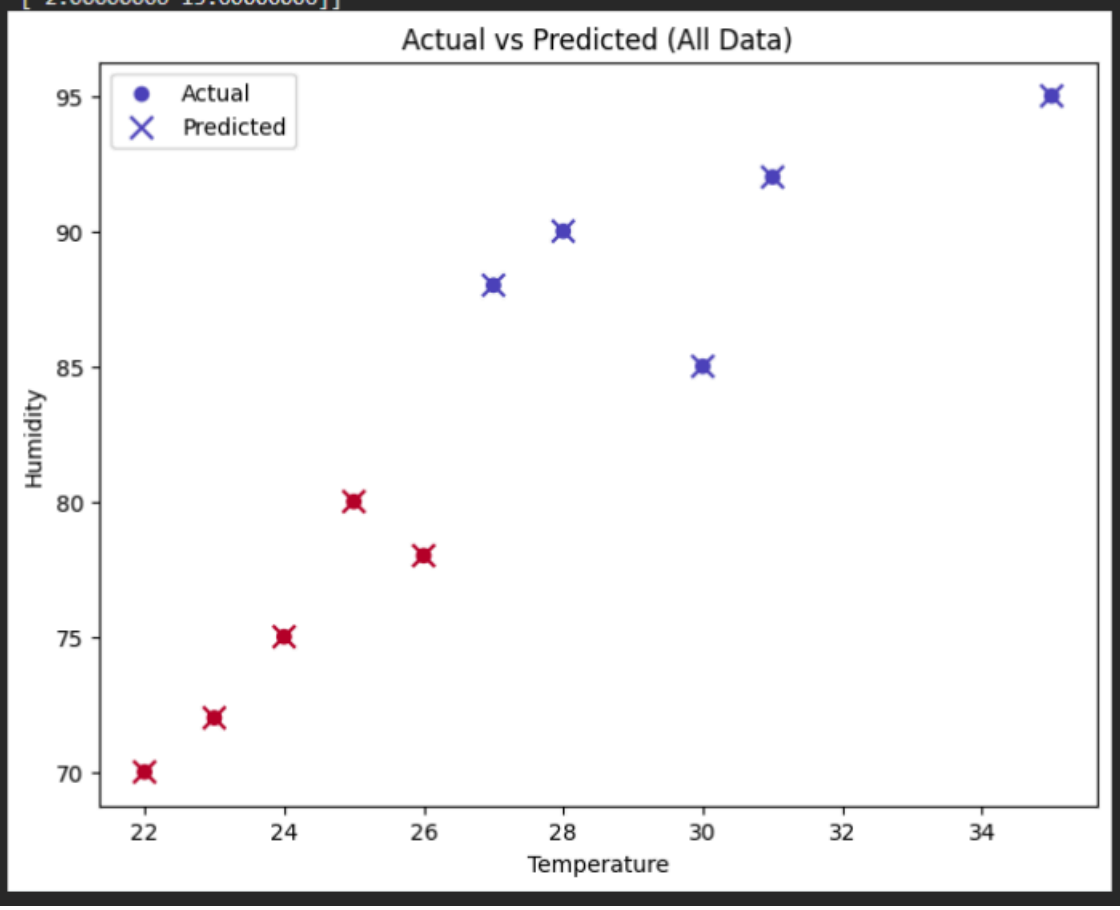
```python
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: X, y, test_size=0.2, random_state=42
)
model = GaussianNB()
model.fit(X_train, y_train)
prediction = model.predict(X_test)
all_prediction = model.predict(X)
print("Accuracy:", accuracy_score(y_test, prediction))
print("\nClass Priors:\n", model.class_prior_)
print("\nMean (theta):\n", model
print("\nVariance:\n", model.var
plt.figure(figsize=(8, 6))
plt.scatter(
    X['Temperature'], X['Humidity'],
    c=y, cmap='coolwarm', marker='o', label='Actual'
)
plt.scatter(
    X['Temperature'], X['Humidity'],
    c=all_prediction, cmap='coolwarm',
    marker='x', s=100, label='Predicted'
)
plt.xlabel("Temperature")
plt.ylabel("Humidity")
plt.title("Actual vs Predicted (All Data)")
plt.legend()
plt.show()
```

C:/Users/jyoth/AppData/Roaming/JetBrains/PyCharm2024.1/scra

model: GaussianNB = GaussianNB()

OUTPUT:

```
Accuracy: 1.0

class priors:
 [0.375 0.625]

Mean (theta) :
 [[30.66666667 89.33333333]
 [24.         75.        ]]

Variance:
 [[10.88888895 17.55555562]
 [ 2.00000006 13.60000006]]
```



EXP N0:5

11249A250_exp1.3.py | 11249A250_expno2.py | 11249A250.expno-3.py | 11249A250.expno-4.py

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
data = pd.read_csv("content/temp_hum_play_data.csv")
le = LabelEncoder()
data['Play'] = le.fit_transform(data['Play'])
X = data[['Temperature', 'Humidity']]
y = data['Play']
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: X, y, test_size=0.2, random_state=42
)
model = GaussianNB()
model.fit(X_train, y_train)
prediction = model.predict(X_test)
all_prediction = model.predict(X)
print("Accuracy:", accuracy_score(y_test, prediction))
print("\nClass Priors:\n", model.class_prior_)
print("\nMean (theta):\n", model.theta_)
print("\nVariance:\n", model.var_)
plt.figure(figsize=(8, 6))
plt.scatter(
    X['Temperature'], X['Humidity'],
    c=y, cmap='coolwarm', marker='o', label='Actual'
)
plt.scatter(
    X['Temperature'], X['Humidity'],
    c=all_prediction, cmap='coolwarm',
    marker='x', s=100, label='Predicted'
```

11249A250.expno-3.py 11249A250.expno-4.py 11249A250_expno-5.py

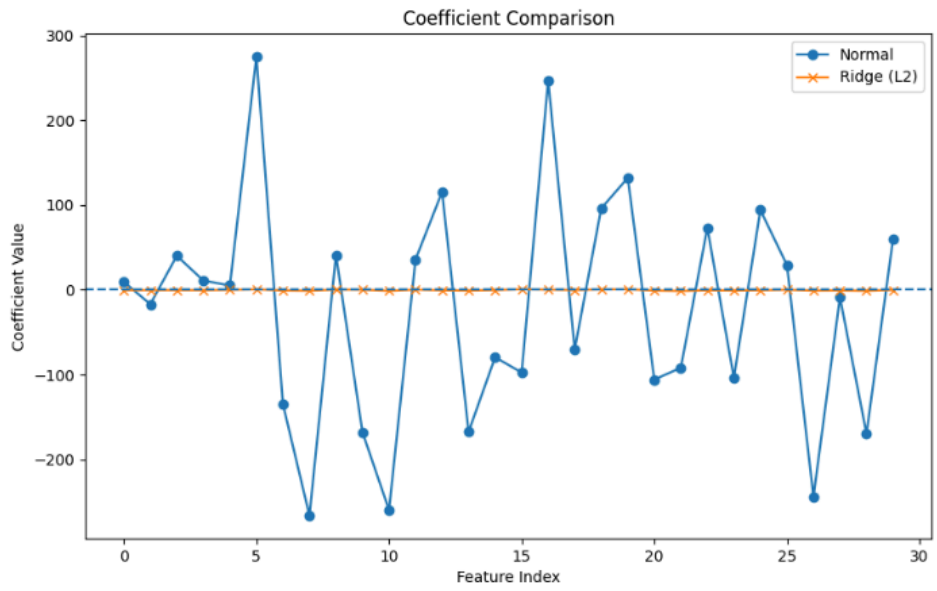
```python
normal.fit(X_train, y_train)
normal_pred = normal.predict(X_test)
normal_acc = accuracy_score(y_test, normal_pred)

# Logistic Regression with L2 (Ridge)
ridge = LogisticRegression(penalty='l2', C=1.0, max_iter=5000)
ridge.fit(X_train, y_train)
ridge_pred = ridge.predict(X_test)
ridge_acc = accuracy_score(y_test, ridge_pred)

# Accuracy
print(f"Normal Accuracy: {normal_acc:.3f}")
print(f"Ridge (L2) Accuracy: {ridge_acc:.3f}")
print("\nNormal Logistic Coefficients:")
print(pd.Series(np.round(normal.coef_[0], decimals: 3), index=features))
print("\nRidge (L2) Coefficients:")
print(pd.Series(np.round(ridge.coef_[0], decimals: 3), index=features))
plt.figure(figsize=(10, 6))
plt.plot( *args: normal.coef_[0], label="Normal", marker='o')
plt.plot( *args: ridge.coef_[0], label="Ridge (L2)", marker='x')
plt.axhline(y=0, linestyle='--')
plt.legend()
plt.title("Coefficient Comparison")
plt.xlabel("Feature Index")
plt.ylabel("Coefficient Value")
plt.show()
```

11249A250_expno-5

OUTPUT:

```
11249A250.expno-3.py    11249A250.expno-4.py    11249A250_expno-5.py
11249A250_expno-5

mean compactness           0.540
mean concavity            -0.801
mean concave points       -1.120
mean symmetry              0.236
mean fractal dimension     0.076
radius error              -1.268
texture error              0.189
perimeter error           -0.611
area error                -0.907
smoothness error          -0.313
compactness error          0.682
concavity error            0.175
concave points error      -0.311
symmetry error             0.500
fractal dimension error    0.616
worst radius              -0.880
worst texture             -1.351
worst perimeter           -0.589
worst area                -0.842
worst smoothness          -0.544
worst compactness          0.016
worst concavity           -0.943
worst concave points      -0.778
worst symmetry            -1.208
worst fractal dimension   -0.157
dtype: float64

Process finished with exit code 0

11249A250_expno-5.py
```

1:1 1,000x600 PNG (24-bit color) 59.09 kB



EXP N0:6

11249A250_exp1.3.py 11249A250_expno2.py 11249A250.expno-3.py 11249A250.expno-4.py

```python
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, StackingClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
iris = datasets.load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: X, y, test_size=0.2, random_state=42
)
rf = RandomForestClassifier(n_estimators=100)
rf.fit(X_train, y_train)
ada = AdaBoostClassifier(n_estimators=100)
ada.fit(X_train, y_train)
estimators = [
    ('rf', RandomForestClassifier()),
    ('svc', SVC(probability=True))
]
stack = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression())
```

11249a250_EXPNO.6

```
C:\Users\jyoth\PycharmProjects\pythonProject\.venv\Scripts\python.exe C:\Users\jyoth\AppData\Roamin
Random Forest Accuracy: 1.0
AdaBoost Accuracy: 0.9333333333333333
Stacking Accuracy: 1.0

Process finished with exit code 0
```

11249a250_EXPNO.6.py

EXP NO:7

50_exp1.3.py | 11249A250_expno2.py | 11249A250.expno-3.py | 11249A250.expno-4.py

```python
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
iris = datasets.load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: X, y, test_size=0.2, random_state=42
)
dt_clf = DecisionTreeClassifier(max_depth=3)
dt_clf.fit(X_train, y_train)
y_pred = dt_clf.predict(X_test)
print("Decision Tree Accuracy:", accuracy_score(y_test, y_pred))
print("Feature Importances:", dt_clf.feature_importances_)
```

11249A250.EXPNO-7

```
C:\Users\jyoth\PycharmProjects\pythonProject\.venv\Scripts\python.exe C:\Users\jyoth\AppD
Decision Tree Accuracy: 1.0
Feature Importances: [0.         0.         0.93462632 0.06537368]

Process finished with exit code 0
```

11249A250.EXPNO-7.py

EXP NO;8

_expno2.py 11249A250.expno-3.py 11249A250.expno-4.py 11249A250_expno-5.py

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris
data = load_iris()
X = data.data
y = data.target
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: X, y, test_size=0.2, random_state=42)
model_linear = SVC(kernel='linear')
model_linear.fit(X_train, y_train)
pred_linear = model_linear.predict(X_test)
acc_linear = accuracy_score(y_test, pred_linear)
print("Linear Kernel Accuracy:", acc_linear)
model_poly = SVC(kernel='poly', degree=3)
model_poly.fit(X_train, y_train)
pred_poly = model_poly.predict(X_test)
acc_poly = accuracy_score(y_test, pred_poly)
print("Polynomial Kernel Accuracy:", acc_poly)
model_rbf = SVC(kernel='rbf')
model_rbf.fit(X_train, y_train)
```

11249A250.EXPNO-8

```
C:\Users\jyoth\PycharmProjects\pythonProject\.venv\Scripts\python.exe C:\Users\jyoth\A
Linear Kernel Accuracy: 1.0
Polynomial Kernel Accuracy: 1.0
RBF Kernel Accuracy: 1.0

Process finished with exit code 0
```

11249A250.EXPNO-8.py

EXP NO:9

```python
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
iris = datasets.load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: X, y, test_size=0.2, random_state=42
)
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
print("K-NN Accuracy:", accuracy_score(y_test, y_pred))
```

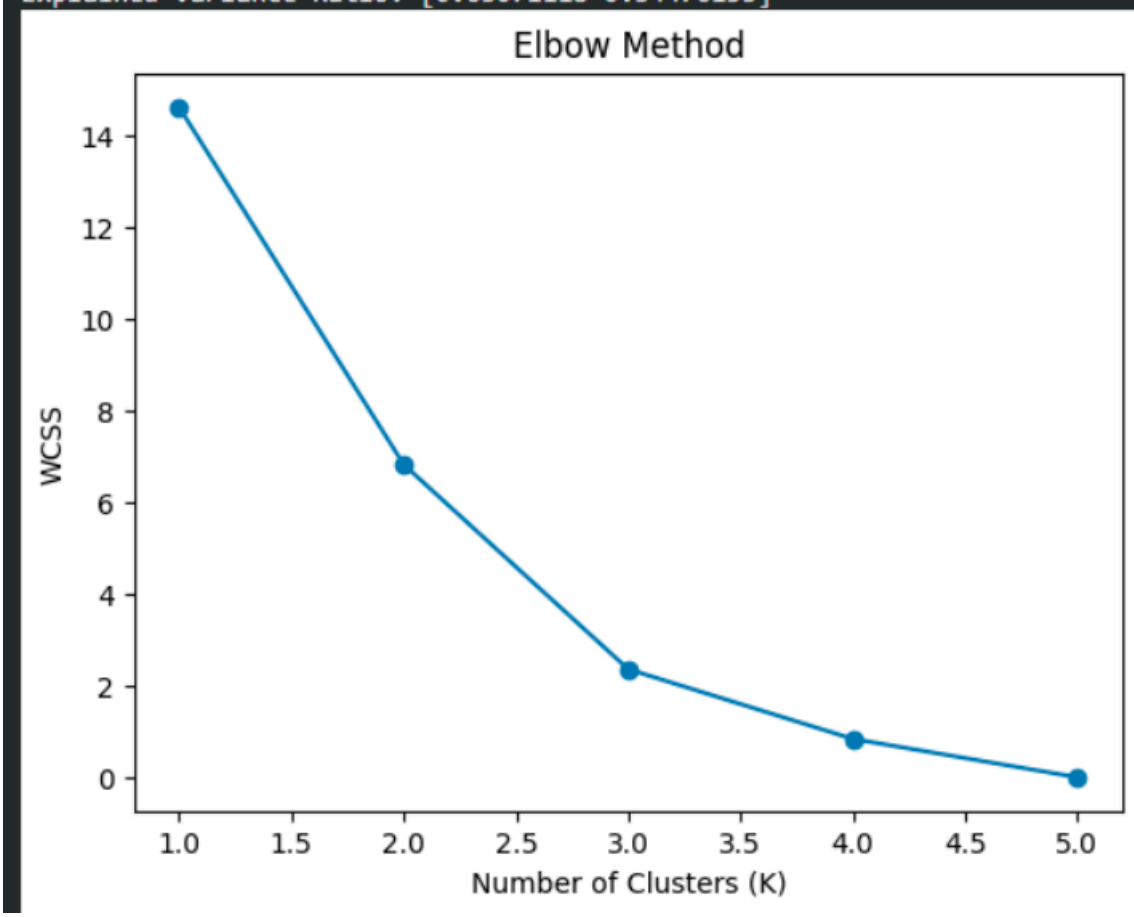
```
C:\Users\jyoth\PycharmProjects\pythonProject\.venv\Scripts\python.exe "C:/Users/jyoth/AppData/Local/JetBrains/PyCharm
K-NN Accuracy: 1.0
Wrote XML report to C:/Users/jyoth/AppData/Local/JetBrains/PyCharm2024.1/coverage/pythonProject$11249A250_EXPNO_9.xml

Process finished with exit code 0
```

EXP NO:10

-4.py | 11249A250_expno-5.py | 11249a250_EXPNO.6.py | 11249A250.EXPNO-7.py | 11249A250.EXPNO-

```python
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
data = pd.read_csv("Mall_Customers.csv")
X = data[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
print("Explained Variance Ratio:", pca.explained_variance_ratio_)
wcss = []
max_k = min(10, len(X_pca))
for i in range(1, max_k + 1):
    kmeans = KMeans(n_clusters=i, random_state=0, n_init=10)
    kmeans.fit(X_pca)
    wcss.append(kmeans.inertia_)
plt.plot( *args: range(1, max_k + 1), wcss, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.show()
optimal_k = 3
kmeans = KMeans(n_clusters=optimal_k, random_state=0, n_init=10)
clusters = kmeans.fit_predict(X_pca)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters, cmap='viridis')
plt.title("K-Means Clustering with PCA")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.show()
```

11249A250-EXPNO-10

OUTPUT:

```
Explained Variance Ratio: [0.63071118 0.34476153]
```